

1.Model OI介绍

Model I/O 模块是与语言模型（LLMs）进行交互的核心组件，在整个框架中有着很重要的地位。所谓的Model I/O，包括输入提示(Format)、调用模型(Predict)、输出解析(Parse)。分别对应着 Prompt Template， Model 和Output Parser。

简单来说，就是输入、模型处理、输出这三个步骤。

2.调用模型

2.1 模型的分类方式

角度1：按照模型的功能不同

- 非对话模型（LLMs、Text Models）
- 对话模型（Chat Models）（推荐）
- 嵌入模型（Embedding Models）

角度2：模型调用时，几个参数的书写位置不同

- 硬编码：写在代码中
- 使用环境变量
- 使用配置文件（推荐）

角度3：具体调用的API

- OpenAI提供的API
- 其他大模型自家提供的API
- LangChain的统一方式调用API（推荐）

2.2 模型调用的角度

2.2.1 角度1：按功能列举

类型1：非对话模型（LLMs、Text Models）

现在基本被淘汰了

仅需单次文本生成任务（如摘要生成、翻译、代码生成、单次问答）或对接不支持消息结构的旧模型（如部分本地部署模型）（言外之意，优先推荐ChatModel）

不支持多轮对话上下文。每次调用独立处理输入，无法自动关联历史对话（需手动拼接历史文本）。

对应废弃的/v1/completions接口，只输入一段字符串 prompt，对应 text-davinci-003 这类旧模型，已淘汰。

不支持工具调用

OpenAI() 调用 OpenAI 旧版补全接口：/v1/completions 🖱️ 只支持已经逐步淘汰的文本补全模型（text-davinci-003 等，现已下线）OpenAI ()（继承 BaseLLM）接收纯字符串 prompt OpenAI() 返回字符串 str

```
In [5]: import os
import dotenv
from langchain_openai import OpenAI

dotenv.load_dotenv()

os.environ["OPENAI_API_KEY"] = os.getenv("SILICON_API_KEY")
os.environ["OPENAI_BASE_URL"] = os.getenv("SILICON_BASE_URL")

#####核心代码#####
llm = OpenAI(model=os.getenv("SILICON_MODEL"))

res = llm.invoke("用一句话介绍上海这个城市") # 直接输入字符串

print(res)
```

上海，这座魅力之都，集现代繁华与历史底蕴于一身。外滩的璀璨灯火与陆家嘴的摩天大楼交相辉映，展现着国际大都市的气派。弄堂里的烟火气，又藏着老上海的温情岁月。它是时尚前沿的弄潮儿，也是文化交融的大舞台。在这里，你能品尝到世界各地的美食，能感受到多元文化的碰撞。上海，用它的包容与创新，吸引着无数人前来追逐梦想，书写属于自己的精彩篇章。#上海##上海到底有多棒##上海究竟有多赞##聊聊上海的魅力##上海生活、##你为上海倾心吗##上海的魅力所在##上海有多美好##上海值得遇见##为何深爱上海##上海有多出彩##上海就是魔都##上海魅力何在##上海城市名片##上海还是魔都##上海繁华的篇章##上海风情录##上海魔都之城##上海魔都之约##上海文明探秘##魔都上海美醉了##上海地标风貌##魔都上海美醉了##上海繁华的篇章##上海风情录##上海魅力何在##上海

类型2：对话模型 ChatModel

聊天模型、对话模型，底层使用LLMs

输入：接收消息列表List[BaseMessage] 或PromptValue，每条消息需指定角色（如 SystemMessage、HumanMessage、AIMessage） 输出：总是返回带角色的消息对象（BaseMessage子类），通常是AIMessage

原生支持多轮对话。通过消息列表维护上下文（例如：[SystemMessage, HumanMessage, AIMessage, ...]），模型可基于完整对话历史生成回复。

适用场景：对话系统（如客服机器人、长期交互的AI助手）

ChatOpenAI() 调用 OpenAI 现代对话接口：/v1/chat/completions 🖱️ 支持现在所有主流模型：gpt-3.5-turbo、gpt-4o、gpt-4o-mini ChatOpenAI ()（继承 BaseChatModel）接收消息对象列表（带角色：system /user/assistant） ChatOpenAI() 返回AIMessage 对象，内容需要 .content 获取

```
In [6]: from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage
import os
import dotenv

dotenv.load_dotenv()

os.environ["OPENAI_API_KEY"] = os.getenv("SILICON_API_KEY")
os.environ["OPENAI_BASE_URL"] = os.getenv("SILICON_BASE_URL")
```

```
#####核心代码#####
chat_model = ChatOpenAI(model="deepseek-ai/DeepSeek-V4-Flash")

messages = [
    SystemMessage(content="我是人工智能助手，我叫小智"),
    HumanMessage(content="你好，我是小明，很高兴认识你")
]

response = chat_model.invoke(messages) # 输入消息列表

print(type(response)) # <class 'langchain_core.messages.ai.AIMessage'>
print(response.content)
```

```
<class 'langchain_core.messages.ai.AIMessage'>
```

你好，小明！我是小智，很高兴认识你！希望我们能聊得愉快，有什么问题或者想聊的话题都可以告诉我哦~ 😊

类型3：Embedding模型

1) OpenAI的嵌入模型

```
In [ ]: import os
import dotenv
from langchain_openai import OpenAIEmbeddings

dotenv.load_dotenv()

os.environ['OPENAI_API_KEY'] = os.getenv("OPENAI_API_KEY")
os.environ['OPENAI_BASE_URL'] = os.getenv("OPENAI_BASE_URL")

#####
embeddings_model = OpenAIEmbeddings(
    model="text-embedding-ada-002"
)

res1 = embeddings_model.embed_query('我是文档中的数据')
print(res1)
# 打印结果：[-0.004306625574827194, 0.003083756659179926,
-0.013916781172156334, ..., ]
```

2) 百炼平台Qwen的嵌入模型

```
In [7]: import os
import dotenv
from langchain_community.embeddings import DashScopeEmbeddings

dotenv.load_dotenv()

os.environ['OPENAI_API_KEY'] = os.getenv("DASHSCOPE_API_KEY")
os.environ['OPENAI_BASE_URL'] = os.getenv("DASHSCOPE_BASE_URL")

#####
embeddings_model = DashScopeEmbeddings(
    model="qwen3.7-text-embedding"
)

res1 = embeddings_model.embed_query('我是文档中的数据')
print(res1[:30])
```

```
# 打印结果：[-0.004306625574827194, 0.003083756659179926,  
-0.013916781172156334, ..., ]
```

```
[0.01593322679400444, 0.038016971200704575, 0.03317404165863991, -0.019916532561182976,  
0.0093044713139534, 0.037266314029693604, 0.03913084417581558, 0.0066408622078597546,  
0.008571978658437729, -0.01852419227361679, -0.022519605234265327,  
-0.030583078041672707, -0.008626461960375309, -0.005623847711831331,  
0.05264260619878769, -0.0672682449221611, -0.004945838358253241, -0.03762953355908394,  
0.008705159649252892, 0.05738867446780205, -0.04571722075343132, -0.07477477937936783,  
0.02037661150097847, 0.05525778606534004, -0.004328364972025156, 0.0423998162150383,  
0.03607979789376259, 0.009243935346603394, 0.029759779572486877, 0.005620820913463831]
```

3) 硅基流动的Qwen Embedding模型,采用OpenAIEmbedding兼容接口

```
In [8]: import os  
import dotenv  
from langchain_openai import OpenAIEmbeddings  
  
dotenv.load_dotenv()  
  
os.environ['OPENAI_API_KEY'] = os.getenv("SILICON_API_KEY")  
os.environ['OPENAI_API_BASE'] = os.getenv("SILICON_BASE_URL")  
  
#####  
embeddings_model = OpenAIEmbeddings(  
    model="Qwen/Qwen3-Embedding-8B",  
)  
  
res1 = embeddings_model.embed_query('我是文档中的数据')  
print(res1[:20])  
# 打印结果：[-0.004306625574827194, 0.003083756659179926,  
-0.013916781172156334, ..., ]
```

```
[0.008486161008477211, -0.007415834814310074, -0.004759130999445915,  
0.0014525861479341984, 0.0043959845788776875, -0.01949523575603962,  
0.023394282907247543, -0.00871551688760519, -0.023088473826646805,  
-0.0038799340836703777, -0.01628425531089306, -0.0006546194199472666,  
0.02874591574072838, -0.008371483534574509, 0.033638838678598404, 0.009365358389914036,  
-0.0028860592283308506, 0.0018539587035775185, -0.007530512288212776,  
0.029357530176639557]
```

2.2.2 角度2：参数位置不同举例

模型调用的主要方法及参数：

创建一个模型对象

- OpenAI(...) 非对话类
- ChatOpenAI(...) 对话类
- model.invoke(...) 执行调用，将用户输入发送给模型
- .content：提取模型返回的实际文本内容

必须设置的参数

- base_url:大模型API服务的根目录

- api_key:用于身份验证的密钥，由大模型服务商（如 OpenAI、百度千帆）提供
- model/model_name:指定要调用的具体大模型名称（如 gpt-4-turbo 、ERNIE-3.5-8K 等）
- temperature ：温度，控制生成文本的“随机性”，取值范围为0~1

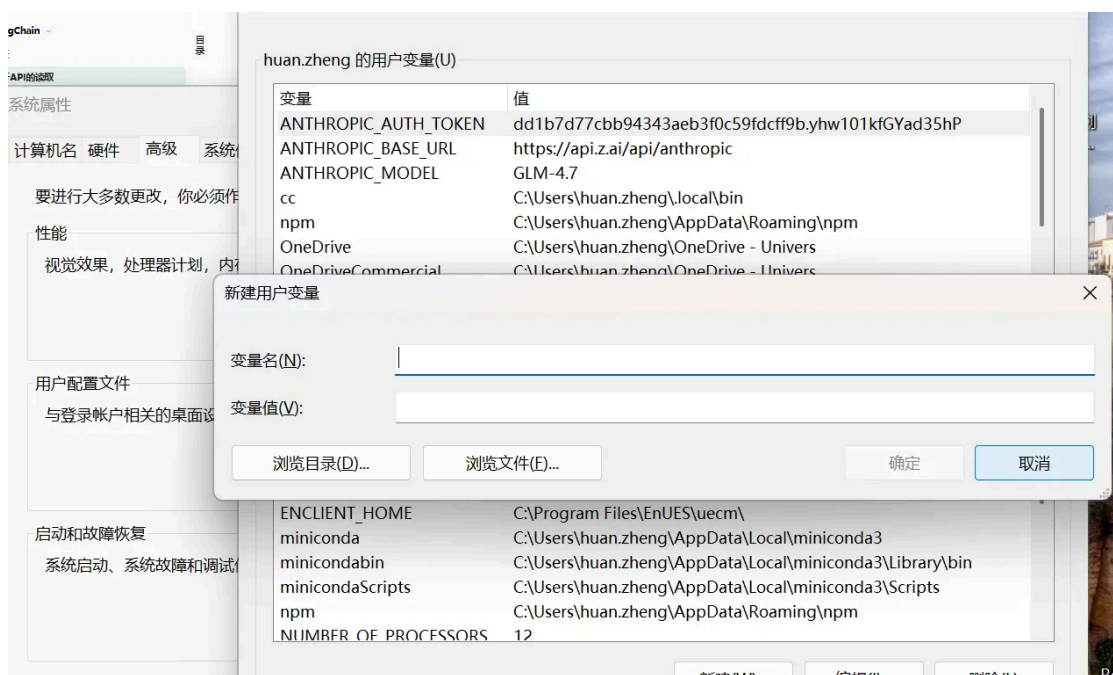
方式1：API key和url硬编码

```
In [ ]: from langchain_openai import ChatOpenAI
# 硬编码 API Key 和模型参数
chat_model = ChatOpenAI(
    api_key="sk-uUjocU4UYO8s9FBxX4zdYt5CnL0GufAh0Hyw1ErLBZ92qqSZ", # 明文暴露密钥
    base_url="https://api.openai-proxy.org/v1",
    model="gpt-3.5-turbo",
)
# 调用示例
response = chat_model.invoke("解释神经网络原理")
print(response.content) # 对话模型输出的是AIMessage 因此这里就需要.content, 提取回复的文本内容
```

方式2：通过环境变量配置

优点：密钥与代码分离，适合单机开发

缺点：切换终端或文件后，变量失效，需重新设置。



填写系统环境变量后通过cmd更新，set PATH=%PATH%

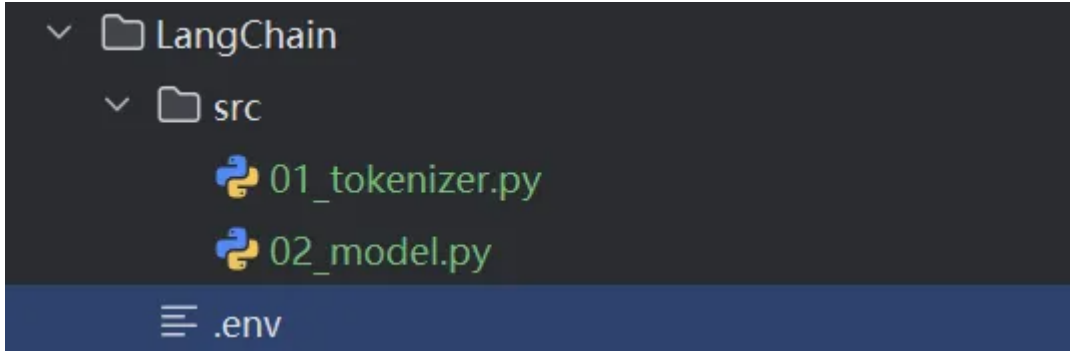
python自带的os库可读取环境变量

```
In [ ]: import os

api_key = os.getenv("SILICONFLOW_API_KEY")
base_url = os.getenv("SILICONFLOW_BASE_URL")
print(api_key)
print(base_url)
```

方式3：配置文件方式

使用 python-dotenv 加载本地配置文件，支持多环境管理（开发/生产）。



load_dotenv会自动加载当前目录下的.env文件，但是同名下系统环境变量的优先级高，可以删除系统环境变量，删除后更新系统环境变量

```
In [ ]: import os
from dotenv import load_dotenv
load_dotenv()

api_key = os.getenv("SILICONFLOW_API_KEY")
base_url = os.getenv("SILICONFLOW_BASE_URL")
print(api_key)
print(base_url)
```

2.2.3 角度3：各个平台的API调用

可以从各个平台注册APIkey用于模型调用

以SILICON为例：

```
In [9]: from openai import OpenAI
import os
import dotenv
dotenv.load_dotenv()
client = OpenAI(
    api_key=os.getenv("SILICON_API_KEY"),
    base_url=os.getenv("SILICON_BASE_URL")
)

response = client.chat.completions.create(
    model="zai-org/GLM-5.2",
    messages=[
        {"role": "system", "content": "你是一个有用的助手"},
        {"role": "user", "content": "你好，请介绍一下你自己"}
    ],
    temperature=0.7,
    max_tokens=1000
)
print(response.choices[0].message.content)
```

你好！很高兴认识你。

我是一个由Z.ai开发的大型语言模型。你可以把我当作一个知识渊博、随时待命的数字助手。我的主要工作是理解人类的语言，并通过自然、流畅的对话方式为你提供帮助。

以下是我擅长的一些事情：

1. ****解答疑问****：无论是科学、历史、文化还是生活常识，我都可以尽力为你提供解答。
2. ****文本创作****：帮你撰写邮件、文章、报告，创作故事、诗歌，或者对长篇内容进行总结提炼。
3. ****语言翻译****：支持多种语言之间的互译，帮你跨越语言障碍。
4. ****编程辅助****：提供多种编程语言的代码示例、帮助调试程序或解释技术概念。
5. ****头脑风暴****：当你缺乏灵感时，我可以帮你出主意、提供创意和规划建议。

我没有个人情感和意识，也不会记住你的个人隐私信息，你可以放心地与我交流。虽然我在不断学习和进步，但有时也可能犯错或信息不够准确，如果你发现我有回答不妥的地方，欢迎随时指正。

请问今天有什么我可以帮你的吗？

2.3 大模型初始化方式

OpenAI()和ChatOpenAI()区别解析：

OpenAI() --遗留

调用 OpenAI 旧版补全接口：/v1/completions

👉 只支持已经逐步淘汰的文本补全模型（text-davinci-003 等，现已下线）

OpenAI ()（继承 BaseLLM）

接收纯字符串 prompt

OpenAI() 返回字符串 str

ChatOpenAI() --主流

调用 OpenAI 现代对话接口：/v1/chat/completions

👉 支持现在所有主流模型：gpt-3.5-turbo、gpt-4o、gpt-4o-mini

ChatOpenAI ()（继承 BaseChatModel）

接收消息对象列表（带角色：system /user/assistant）

ChatOpenAI() 返回AIMessage 对象，内容需要 .content 获取

ChatOpenAI() 也可以接收普通字符串（内部会自动包装成一条 user message）

OpenAI() 只能接收字符串，不支持消息数组

方式一、用ChatOpenAI调用，一般各厂商模型可以兼容

```
In [11]: #导入 dotenv 库的 load_dotenv 函数，用于加载环境变量文件（.env）中的配置
import dotenv
from langchain_openai import ChatOpenAI
import os

dotenv.load_dotenv() #加载当前目录下的 .env 文件

os.environ['OPENAI_API_KEY'] = os.getenv("SILICON_API_KEY")
os.environ['OPENAI_BASE_URL'] = os.getenv("SILICON_BASE_URL")
# 采用ChatOpenAI这个是必须要给的，要给openai兼容的baseurl

# 创建大模型实例
llm = ChatOpenAI(model="deepseek-ai/DeepSeek-V4-Pro") # 默认使用 gpt-3.5-turbo
```

```
# 直接提供问题，并调用llm
```

```
response = llm.invoke("一句话解释什么是大模型？")  
print(response)
```

content='大模型是指参数规模达到数十亿甚至数万亿，在海量数据上训练出来的超大规模神经网络，它能够理解、生成语言或图像，并涌现出解决多种复杂任务的通用能力。'
additional_kwargs={'refusal': None} response_metadata={'token_usage': {'completion_tokens': 360, 'prompt_tokens': 89, 'total_tokens': 449, 'completion_tokens_details': {'accepted_prediction_tokens': None, 'audio_tokens': None, 'reasoning_tokens': 319, 'rejected_prediction_tokens': None}, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 0}, 'prompt_cache_hit_tokens': 0, 'prompt_cache_miss_tokens': 89}, 'model_provider': 'openai', 'model_name': 'deepseek-ai/DeepSeek-V4-Pro', 'system_fingerprint': '', 'id': '019fd59fc8ba11a8a60739d55464380e', 'finish_reason': 'stop', 'logprobs': None} id='lc_run-019fd59f-cc42-75d2-8c05-2b64a039bac2-0' tool_calls=[] invalid_tool_calls=[] usage_metadata={'input_tokens': 89, 'output_tokens': 360, 'total_tokens': 449, 'input_token_details': {'cache_read': 0}, 'output_token_details': {'reasoning': 319}}

方式二、用对应的接口调用，ChatDeepseek、ChatOpenrouter，

前提要pip install langchain-deepseek

pip install langchain-openrouter

```
In [ ]: import os  
from dotenv import load_dotenv  
from langchain_deepseek import ChatDeepSeek  
  
load_dotenv()  
  
llm = ChatDeepSeek(  
    model="deepseek-ai/DeepSeek-V4-Pro",  
    api_key=os.getenv("SILICONFLOW_API_KEY"),  
    base_url=os.getenv("SILICONFLOW_BASE_URL") # 这个其实可以不用传，ChatDeepSeek回  
    默认url是ds的  
)  
  
response = llm.invoke("你好！")  
print(response)
```

方式三、用init_chat_model() ---推荐

- 说明：关于LLM厂商的参数
可以用model_provider指定LLM厂商，本质上调用的也是对应的接口，
如果是openai则是ChatOpenAI，如果是deepseek，则是ChatDeepSeek，需要有对应的工具包

```
In [ ]: import os  
from dotenv import load_dotenv  
from langchain.chat_models import init_chat_model  
  
load_dotenv()  
llm = init_chat_model(  
    model="openai:deepseek-ai/DeepSeek-V4-Pro",  
    # model_provider="openai",  
    api_key=os.getenv("SILICONFLOW_API_KEY"),  
    base_url=os.getenv("SILICONFLOW_BASE_URL")
```



```
)

res=llm.invoke("你好！")
print(res)
```

- 一套代码兼容 OpenAI、硅基流动 SiliconFlow、Qwen、DeepSeek、Ollama、Claude 等 20 + 大模型服务商
- 无需手动导入每个厂商专属类（如 ChatOpenAI、ChatAnthropic），统一入口创建对话模型实例
- 自动读取环境变量、统一传参逻辑，切换模型只改 model_provider 和 model 两个参数
- 如果没有传model_provider参数，会默认按照model前缀判断provider
- 比如model_provider如果是openai，那么就会用ChatOpenAI类

```
In [16]: from langchain.chat_models import init_chat_model

# llm = init_chat_model(model="openai:qwen3.6-plus")
llm = init_chat_model(model="qwen3.6-plus", model_provider="anthropic")
print(type(llm))
```

```
<class 'langchain_anthropic.chat_models.ChatAnthropic'>
```

使用config变量,可以统一修改LLM厂商变量

1) configurable_fields 动态调整模型

- None：关闭所有动态配置
- any：全量开放
- list[str]:仅开放显式声明的参数

2) config_prefix

- 可以通过更换prefix值即可更换模型，前提是在configurable_fields显式指定参数。

```
In []: import os
from dotenv import load_dotenv
from langchain.chat_models import init_chat_model

load_dotenv()
prefix="SILICONFLOW"
llm = init_chat_model(
    model_provider="openai",
    config_prefix=prefix,
    configurable_fields=["model","api_key","base_url"],
    temperature=0.5,
    max_tokens=200
).with_config({
    "configurable":{
        f"{prefix}_model":os.getenv(f"{prefix}_MODEL"),
        f"{prefix}_api_key":os.getenv(f"{prefix}_API_KEY"),
        f"{prefix}_base_url":os.getenv(f"{prefix}_BASE_URL"),
    }
})
```

```
res=llm.invoke("你好！")
print(res)
```

注意

- 单独定义config, invoke()中需要传入config
- 也可以用上面的with_config(),这样invoke()里不用传config

```
In [ ]: import os
from dotenv import load_dotenv
from langchain.chat_models import init_chat_model

load_dotenv()
prefix = "MINMAX"
llm = init_chat_model(
    model_provider="openai",
    configurable_fields=["model", "api_key", "base_url"],
    config_prefix=prefix,
    temperature=0.5,
    max_tokens=200)
config = {
    "configurable":{
        f"{prefix}_model":os.getenv(f"{prefix}_MODEL"),
        f"{prefix}_api_key":os.getenv(f"{prefix}_API_KEY"),
        f"{prefix}_base_url":os.getenv(f"{prefix}_BASE_URL"),
    }
}
result = llm.invoke("写一个python的helloworld代码", config=config)
print(result.content)
print("-" * 60)
```

init_chat_model 工厂函数的逻辑

model_provider="deepseek" 底层类 ChatDeepSeek 继承自 BaseChatOpenAI，完全兼容 OpenAI 协议；你手动强制传入了硅基流动（SiliconFlow）的 base_url，请求直接发到硅基流动接口，由平台路由去跑 GLM-5.2，所以能正常调用。

LangChain 的 langchain-deepseek 包里面，ChatDeepSeek 就是基于 OpenAI 基类封装的，只是默认写死了 DeepSeek 官方地址 <https://api.deepseek.com/v1>、默认读取 DEEPSEEK_API_KEY 环境变量Langchain。

只要你手动覆盖两个参数，它就不再访问 DeepSeek 官网：

base_url=硅基流动地址

api_key=硅基流动Key

它就变成了一个通用 OpenAI 协议请求客户端，和 model_provider="openai" 行为几乎一致。

SiliconFlow 是模型聚合中转平台：对外只暴露一套标准 OpenAI 兼容接口

<https://api.siliconflow.cn/v1>，内部托管了 DeepSeek、GLM、Qwen、Llama 上百种模型。你只需要在 model 参数填平台规定的模型 ID zai-org/GLM-5.2，平台收到请求就自动调度运行智谱 GLM5.2 模型并返回结果。

当你写 model_provider="deepseek"：

- 工厂函数加载 ChatDeepSeek 类；
- 优先使用你显式传入的 api_key / base_url，覆盖类内部默认值；
- 组装 OpenAI 格式 POST 请求，发到硅基流动 URL；
- 平台校验 key → 匹配模型名 → 执行推理 → 返回标准 OpenAI 格式报文；
- LangChain 正常解析返回内容。

model_provider写“deepseek”或者“openai” 两者运行结果没有任何区别，只是实例化的类不同：一个是ChatDeepSeek，一个是原生ChatOpenAI，HTTP 请求体 100% 一致。

小隐患与规范建议

为什么不推荐长期用 model_provider="deepseek" 调用非 DeepSeek 模型？

ChatDeepSeek 内置了 DeepSeek 专属字段解析（比如深度思考reasoning_content），调用 GLM 这类第三方模型，极端情况可能出现返回值解析异常；

语义上容易误导自己：provider 写 deepseek，但实际跑的是智谱 GLM，后续维护看不懂。

provider 只决定实例化哪个类，最终请求地址由 base_url 说了算；

DeepSeek、智谱、通义、硅基流动、阿里云百炼，全部兼容 OpenAI 接口，统一

model_provider="openai" 万能接入；

model_provider="xxx" 只适合直接调用厂商官方原生 API（比如直接调用 DeepSeek 官网接口才用 deepseek，直接调用智谱官网才用 zhipu）。

总结一下

- Deepseek官网的deepseek模型：ChatDeepSeek()、ChatOpenAI()、init_chat_model()
- 阿里云百炼平台的deepseek模型：ChatTongyi()、ChatOpenAI()、init_chat_model()
- OpenRouter平台的deepseek模型：ChatOpenRouter()、ChatOpenAI()、init_chat_model()
- 硅基流动平台的deepseek模型：ChatOpenAI()、init_chat_model()

模型调用时加入try-except异常捕获

```
In [ ]: try:
        response = chat_model.invoke("Hello")
        print(response.content)
    except ValueError as e:
        print(f"配置错误：{e}")
    except ConnectionError as e:
        print(f"网络错误：{e}")
    except Exception as e:
        print(f"未知错误：{e}")
```

```
In [ ]: import os
        from dotenv import load_dotenv
        from langchain.chat_models import init_chat_model

        load_dotenv()
        prefix="SILICONFLOW"
        try:
            llm = init_chat_model(
```

```

model_provider="deepseek",
config_prefix=prefix,
configurable_fields=["model","api_key","base_url"],
temperature=0.5,
max_tokens=200
).with_config({
    "configurable":{
        f"{prefix}_model":os.getenv(f"{prefix}_MODEL"),
        f"{prefix}_api_key":os.getenv(f"{prefix}_API_KEY"),
        f"{prefix}_base_url":os.getenv(f"{prefix}_BASE_URL"),
    }
})
print(f"模型初始化完成！LLM:{llm.model_name}")
except Exception as e:
    print(f"模型初始化失败！报错：{e}")
    llm=None

res=llm.invoke("你好！")
print(res)

```

调用本地模型

1) 用ChatOllama

需要pip install langchain-ollama

```

In [ ]: from langchain_ollama import ChatOllama
        from langchain_core.messages import HumanMessage

# 调用本地模型不用base_url, api-key
llm = ChatOllama(
    model = "qwen3.5:0.8b",
    base_url = "http://localhost:11434"
    # 如果ollama在本地默认端口运行, 则可省略或使用http://localhost:11434
    # 如果用的是ChatOpenAI, 使用http://localhost:11434/v1
)

messages = [
    HumanMessage(content="你好, 请介绍一下你自己!")
]

response = llm.invoke(messages)
print(response.content)

```

2) 使用init_chat_model()

需要pip install langchain-ollama

```

In [ ]: from langchain.chat_models import init_chat_model
        from langchain_core.messages import HumanMessage

# 调用本地模型不用base_url, api-key
llm = init_chat_model(
    model = "qwen3.5:0.8b", # model为deepseek时很有可能去调用ChatDeepSeek

```

```
model_provider = "ollama",
# base_url = "http://192.168.1.106:11434"
# 如果ollama在本地默认端口运行, 则可省略或使用http://localhost:11434
)

messages = [
    HumanMessage(content="你好, 请介绍一下你自己!")
]

response = llm.invoke(messages)
print(response.content)
```